

Regressão Linear e Mínimos Quadrados

Aula 6 – Da Intuição Algorítmica à Implementação

Prof. Rodrigo

Instituto Federal de Educação, Ciência e Tecnologia de São Paulo - IFSP

Semestre Letivo

Objetivos de Aprendizagem

Objetivo Principal

O foco é entender **como o algoritmo aprende** e **como depurar (debug) modelos matemáticos**.

Competências Computacionais:

- **Mapeamento de Dados:** Compreender como os atributos (entradas) são multiplicados pelos pesos matemáticos para gerar as saídas.
- **Otimização:** Entender, de forma visual e algorítmica, como o método de Mínimos Quadrados (MQO) encontra a melhor solução.

Engenharia e Qualidade (QA):

- **Condições de Contorno:** Conhecer as regras (Gauss-Markov) que nossos dados precisam respeitar para o sistema não falhar.
- **Depuração:** Analisar gráficos de resíduos para encontrar erros nos dados (como heterocedasticidade e outliers).

1. Introdução: O Sistema de Previsão

Em engenharia de software, criamos funções onde a entrada processa uma saída. Em Aprendizado de Máquina, nós **descobrimos a função** a partir de dados históricos.

- **A Função Ideal (Mundo Real):** A resposta real (y) que queremos prever (ex: Preço de um Imóvel) tem uma lógica mais um fator aleatório (*ruído*):

$$y = f(x) + \epsilon$$

- **A Aproximação do Sistema (Nosso Código):** Assumimos que o preço é uma soma ponderada (combinação linear) dos atributos (x):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

- **Os Pesos (β):** São as “variáveis de configuração” do nosso algoritmo. O β_1 diz exatamente quanto o preço do imóvel sobe se adicionarmos $1m^2$ (x_1).

2. Traduzindo para Estruturas de Dados (Matrizes)

Para processar milhares de registros ao mesmo tempo sem usar *loops for*, o computador usa Álgebra Linear (operações com *Arrays*).

A Equação Vetorial do Sistema

$$Y = X\beta + \epsilon$$

Lendo a equação como programador:

- Y : O *Array* (1D) com os alvos (Preços reais).
- β : O *Array* com os pesos que o modelo precisa aprender.
- ϵ : O *Array* de erros (o que o modelo não consegue explicar).

Matriz X (Nosso *DataFrame* do *Pandas*):

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & x_{N2} & \dots & x_{Np} \end{bmatrix}$$

Cada linha é um cliente/imóvel. A coluna de 1s serve para o algoritmo calcular o intercepto (β_0).

3. Como o computador "enxerga" o ajuste?

A regressão linear não é apenas passar uma reta em um gráfico 2D.

- **O Espaço dos Dados:** Imagine que cada coluna do nosso banco de dados (Tamanho, Quartos, etc) é um vetor apontando para uma direção matemática.
- **O Subespaço Coluna $Col(X)$:** Esses atributos formam um “chão” (um plano). O algoritmo só consegue fazer previsões que existam dentro desse plano.
- **O Problema:** Os preços reais (Vetor Y) não estão perfeitamente deitados nesse plano, pois os dados reais têm ruído/imperfeições.

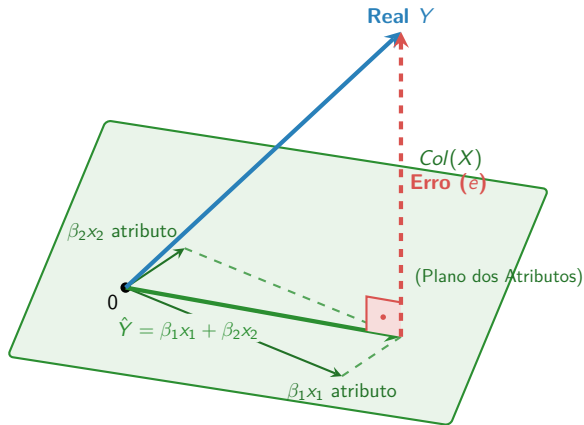
4. Projeção Ortogonal: Achando a melhor solução

Se o preço real Y está “flutuando” fora do plano do modelo, como o algoritmo calcula a melhor previsão $\hat{Y}$ possível?

A Sombra Perfeita

O algoritmo encontra a projeção ortogonal (a sombra) de Y sobre o plano dos atributos X .

- Matematicamente, a menor distância entre o ponto real e o plano é uma linha reta (perpendicular/ortogonal).
- Esse “fio de prumo” é exatamente o nosso **Erro (Resíduo)**.



5. Módulo 2 – A Lógica dos Mínimos Quadrados (MQO)

Todo algoritmo de Aprendizado de máquina precisa de uma **Função de Custo (Loss Function)**. Ela mede o quão “ruim” o modelo está.

A Função RSS (Soma dos Erros ao Quadrado)

$$RSS(\beta) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N (y_i - x_i^T \beta)^2$$

- **Por que ao quadrado?** Para impedir que um erro de -50 cancele um erro de +50, e para penalizar previsões absurdamente distantes do real.
- O objetivo do treinamento (função `fit()`) é encontrar o Array de pesos β que resulta no **menor RSS possível**.

6. Encontrando o Mínimo da Função (Otimização)

Como achamos o menor erro? Em cálculo, sabemos que o ponto mais baixo de uma curva (como uma bacia parabólica) acontece quando a **derivada é igual a zero**.

Aplicando regras de derivação matricial (semelhante a derivar um polinômio, mas com Arrays) na função de erro, chegamos a:

$$\frac{\partial RSS(\beta)}{\partial \beta} = -2X^T Y + 2(X^T X)\beta$$

Ao forçar essa derivada a ser ZERO, nós avisamos a CPU: *“Encontre os pesos onde a curva de erro para de cair e atinge o fundo”*.

7. As Equações Normais

Igualando a derivada anterior a zero, chegamos a um sistema linear simples, que é o que o computador resolve nos bastidores:

Equações Normais

$$(X^T X)\beta = X^T Y$$

- **Garantia Matemática:** Como a função de Erro Quadrático forma um “paraboloide perfeito” (convexa), a solução encontrada aqui não é uma aproximação, é o **mínimo global**. É impossível existir pesos melhores para aqueles dados lineares.

8. A Solução Analítica (O que a biblioteca executa)

Se tratarmos a matriz $(X^T X)$ como um número, bastaria “passar dividindo”. Em matrizes, passamos multiplicando pela **matriz inversa**.

Fórmula do Treinamento de Regressão Linear

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

- **Traduzindo para Código:** Quando você chama `model.fit(X, Y)` no Scikit-Learn, a CPU (na implementação mais básica) executa exatamente:

```
beta = np.linalg.inv(X.T @ X) @ X.T @ Y
```

9. Quando o sistema falha? (Singularidade Matricial)

A solução analítica exige calcular a inversa de $(X^T X)$. Na programação, se tentarmos inverter uma matriz singular, recebemos como resposta uma *Exception* (LinAlgError).

O que causa esse erro nos seus dados?

- **Multicolinearidade Perfeita:** Se você tiver colunas duplicadas ou que são combinações óbvias (Ex: Coluna A = Temperatura em Celsius; Coluna B = Temp. em Fahrenheit). Os *atributos* precisam ser independentes.
- **Amostra Insuficiente ($N < p$):** Se você tem mais colunas (atributos) do que linhas (amostras), o sistema tem infinitas soluções e o método quebra. (Será necessário usar Regularização de dados).

10. Módulo 3 — Teorema de Gauss-Markov

Por que a Regressão Linear Clássica ainda é tão usada na indústria se temos Redes Neurais?

O "Selo de Qualidade" BLUE

Sob certas regras (Gauss-Markov), o estimador de Mínimos Quadrados é comprovadamente **BLUE** (*Best Linear Unbiased Estimator*) ou seja o melhor estimador linear não enviesado.

- **Linear (L):** Relação direta entre entrada e saída.
- **Não Viesado (U - Unbiased):** Na média, o algoritmo não “arrasta” as previsões para cima ou para baixo.
- **Melhor (B - Best):** Dentre todos os algoritmos lineares, este é o que tem a **menor variância** (as previsões são mais estáveis e confiáveis).

1. Linearidade nos Parâmetros

Para o modelo funcionar e ter o selo BLUE, algumas suposições sobre a base de dados precisam ser atendidas.

- **O que a máquina exige:** O modelo deve ser linear nos pesos (β), mas NÃO obrigatoriamente nos *atributos* de entrada (x).
- **Engenharia de atributos é permitido:** Você pode criar colunas não-lineares no DataFrame e ainda usar Regressão Linear!

$$y = \beta_0 + \beta_1(x_1^2) + \beta_2 \ln(x_2) + \epsilon$$

- **Proibido:** Equações onde o **peso** é expoente, ex: $y = e^{\beta_1 x_1}$. (Nesse caso, precisaríamos de uma rede neural ou solver não-linear).

2. Exogeneidade (Variável Omitida)

Média Condicional Nula

O erro do algoritmo não deve ter relação oculta com as variáveis de entrada. ($E[\epsilon|X] = 0$)

- **O erro da Variável Omitida:** Imagine prever o salário de desenvolvedores baseando-se apenas em “Anos de Estudo”, omitindo os “Anos de Experiência” e a “Linguagem de Programação”.
- Como o modelo ignora esses *atributos*, o impacto delas vaza para a margem de erro. O algoritmo compensará isso inflando o peso de “Anos de Estudo” artificialmente, resultando em um modelo com viés (subajustado).

3. Homocedasticidade

A palavra é difícil, mas o conceito é de Qualidade de Software (QA): **O comportamento do erro deve ser estável em todo o sistema.**

Variância Constante

$Var(\epsilon_i|x_i) = \sigma^2 \quad \forall i$ (A margem de erro não muda dependendo do dado de entrada).

- **Quando quebra (Heterocedasticidade):** Tentar prever gastos com alimentação. Pessoas que ganham R\$2.000 gastam valores previsíveis. Pessoas que ganham R\$50.000 variam absurdamente (alguns comem marmita, outros vão a restaurantes Michelin).
- **Consequência:** O algoritmo de Aprendizado de máquina continua acertando na média, mas os “intervalos de confiança” (margem de erro) reportados pelo sistema deixam de ser confiáveis.

4. Ausência de Autocorrelação

Os erros cometidos na predição de um registro não podem ajudar a prever o erro do próximo registro do DataFrame.

Erros Independentes

$$\text{Cov}(\epsilon_i, \epsilon_j | X) = 0 \quad \forall i \neq j$$

- **Onde costuma gerar erro:** Dados sequenciais (Séries Temporais). Exemplo: Se o modelo errou o clima prevendo a temperatura para menos hoje, a chance de ele errar para menos amanhã é muito alta (porque frentes frias duram dias).
- **Consequência:** Novamente, o sistema reportará uma métrica de precisão ilusória e enganosa.

15. Provando que a Regressão Linear não tem Viés

A matemática garante que, se você treinar seu modelo com boas amostras de dados (respeitando as regras anteriores), **ele acerta na média estatística**.

Dedução (Demonstração da Propriedade Unbiased)

Lembre-se que: $\hat{\beta} = (X^T X)^{-1} X^T Y$. Como sabemos que $Y = X\beta + \epsilon$:

$$\hat{\beta} = (X^T X)^{-1} X^T (X\beta + \epsilon)$$

$$\hat{\beta} = \beta + (X^T X)^{-1} X^T \epsilon$$

Aplicando a expectativa condicional (a média esperada):

$$E[\hat{\beta}|X] = \beta + (X^T X)^{-1} X^T \cdot E[\epsilon|X]$$

Como a média do erro ($E[\epsilon|X]$) é zero (Regra 2):

$$E[\hat{\beta}|X] = \beta$$

Conclusão: Seu algoritmo aprenderá os pesos no longo prazo.

16. Módulo 4 – Depuração e Avaliação do Modelo

Treinamos o modelo, mas como garantimos a entrega de um software de qualidade para o cliente?
Medindo o grau de erro.

- **A Decomposição da Variância:** Pense nisso como dividir de onde vêm os “problemas” dos dados.

$$SST = SSR + SSE$$

Onde:

- **SST (Variabilidade Total):** O caos natural dos preços na vida real.
- **SSR (Explicado pelo Modelo):** A parte do caos que nosso algoritmo conseguiu decifrar e prever com sucesso.
- **SSE (Soma dos Resíduos):** A parte do caos que o algoritmo **não conseguiu** entender (O ERRO).

17. A Métrica: R^2 (R-Quadrado)

O R^2 é a métrica principal para medir a qualidade do modelo preditivo.

Fórmula do Score

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

- **Tradução Simples:** *"De todo o caos existente na base de dados, qual a porcentagem o nosso sistema conseguiu explicar?"*
- **Escala de 0 a 1:**
 - $R^2 = 1.0$: Ajuste perfeito. Score de 100%. Nenhum erro matemático.
 - $R^2 = 0.0$: O modelo é inútil, tão eficiente quanto apenas sortear a média dos valores.

18. Cuidado com Modelos Mentirosos: O R^2 Ajustado

Em engenharia de dados, existe a tentação de adicionar centenas de atributos (colunas inúteis) no modelo para ver se o score melhora.

O Truque Matemático

A matemática do R^2 tem um defeito: Toda vez que você adiciona uma coluna no DataFrame, o R^2 **sempre** sobe ou empata. Ele nunca cai. Isso leva ao **Overfitting**.

- **A Correção em Produção (R^2 Ajustado):** Adiciona uma penalidade (como um imposto) para cada variável (p) que você coloca no modelo.

$$R_{Ajustado}^2 = 1 - (1 - R^2) \frac{N - 1}{N - p - 1}$$

Se a nova coluna for inútil, a penalidade falará mais alto e a nota geral do seu modelo **vai cair**.

19. Depuração 1: Resíduos vs. Valores Ajustados

O gráfico de dispersão que cruza “O Erro” (e_i) com a “Previsão” ($\hat{y}_i$) é a nossa tela principal de diagnóstico.

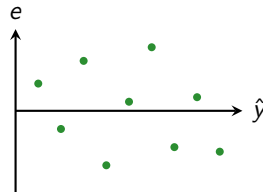
Padrão Esperado (Bom):

- Nuvem aleatória, parece ruído estático de TV. A Regressão Linear funcionou perfeitamente.

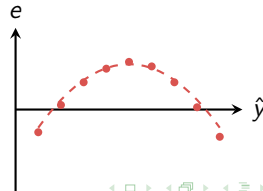
Padrão Não Linear (Erro):

- Formato de arco ou "U".
- **Refatoração Necessária:** Seus dados não são lineares. Você precisa adicionar variáveis ao quadrado (x^2) na matriz de *atributos*.

Cenário Ideal (Ruído Aleatório)



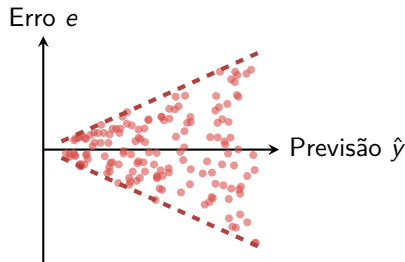
Cenário com Falha (Padrão Não-Linear)



20. Depuração 2: Identificando Heterocedasticidade

Como identificar graficamente a quebra da hipótese de erro constante?

- **O Efeito Funil (*Fan Shape*):** Os pontos abrem como um cone à medida que a previsão cresce.
- **O que significa?** O seu software erra por 10 reais para imóveis baratos, mas erra por 50 mil reais para imóveis de luxo. A precisão se degrada.
- **Refatoração:** O Cientista de Dados corrige isso aplicando uma escala logarítmica no preço original (Y) antes do treinamento.



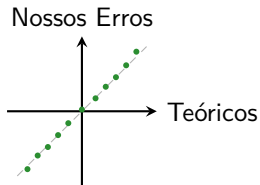
21. Depuração 3: Gráfico Q-Q Normal

Para validarmos estatisticamente se os pesos (W) calculados são realmente bons e se as variáveis não são inúteis (Testes P-Value), o nosso Erro (Bug) precisa seguir uma **Distribuição Normal (Curva de Sino)**.

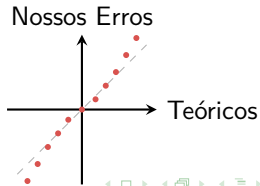
Gráfico Q-Q (Quantile-Quantile):

- Cruza os seus erros com o que seria o erro "matematicamente perfeito"(normal).
- **Aprovação (Cima):** Se os pontos formarem uma diagonal de 45º, os erros são Gaussianos.
- **Rejeição (Baixo):** Curvas em formato de "S"apontam caudas pesadas e *outliers*.

Cenário Ideal (Erros Normais)



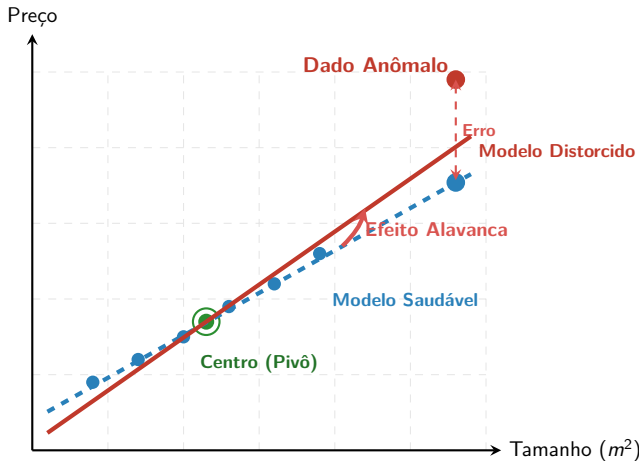
Cenário com Falha (Formato de "S")



22. Registros Perigosos no Banco de Dados (Leverage/Outliers)

Nem todas as linhas do seu banco de dados importam igualmente para o modelo.

- **Alavancagem (Leverage):** Um único imóvel absurdamente fora do padrão (Ex: Mansão isolada num bairro popular) atrai a reta de predição inteira para si como um ímã (Torque).
- **Refatoração:** Filtros robustos. Detecte e exclua (ou aplique *clipping*) nesses outliers ANTES de rodar o `model.fit()`.



23. Boas Práticas: Escalonamento e Performance da CPU

Lembra da matriz que sofria problemas matemáticos de inversão (Módulo 2)? Existe um simples detalhe de código que previne problemas de cálculo.

Sem padronização de atributos:

- Se você cruzar variáveis pequenas (Idade: 20 a 50) com variáveis imensas (Salário: R\$3.000 a R\$50.000), o sistema linear fica muito instável.
- Isso causa erros graves de arredondamento de casas decimais (overflow de floats) internamente na CPU durante a matemática.

A solução (StandardScaler):

- Sempre normalize os dados antes! Ao centralizar tudo (média 0, desvio padrão 1), o “modelo matemático” fica homogêneo e circular.
- Garantia de velocidade de execução de código e nenhuma falha nas matrizes!

Nesta sessão conectamos conceitos complexos diretamente ao código:

- ➊ **Mínimos Quadrados / Equação Normal:** É a função algorítmica de cálculo de custo para minimizar a margem de erro.
- ➋ **Gauss-Markov (BLUE):** Garantia estrutural de que nosso banco de dados tem a qualidade exigida para um modelo perfeitamente não-viesado.
- ➌ **R-Quadrado (R^2 e R^2 Ajustado):** Teste de Qualidade (QA) para medir se o modelo merece subir para o servidor em produção.
- ➍ **Heterocedasticidade / Autocorrelação:** Erros escondidos nos dados reais que exigem refatoração (*feature engineering*).

- ❶ **GÉRON, Aurélien.** *Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow*. Alta Books.
 - Capítulo 4: Mostra a codificação direta por trás das equações de regressão e matrizes SVD.
- ❷ **Documentação Oficial do Scikit-Learn (scikit-learn.org).**
 - Material rico em tutoriais de código pragmáticos e gráficos demonstrativos sobre a matemática dos regressores.
- ❸ **IZBICKI, Rafael; SANTOS, Tiago.** *Aprendizado de Máquina: Uma Abordagem Estatística*. São Carlos.
 - Capítulo 2: Regressão.